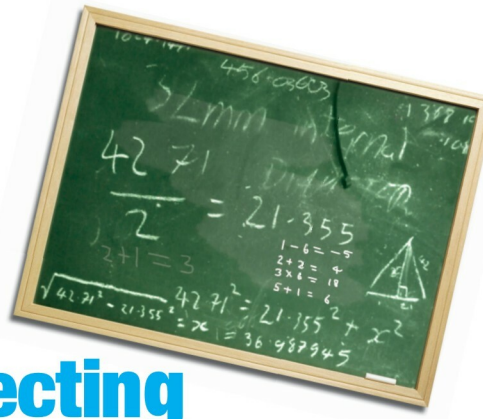

Dissecting Register Allocation

Register allocation is a vital part of compiler development. Programmers don't have to bother about how many physical registers are available since they code as if an infinite number is at their disposal. Neither do they worry about how they will be allocated to operations in the program. However, the number of physical registers for any processor is limited, so the compiler has to judge what registers to allocate to perform all the program's operations. This task is done by the Register Allocation phase of the compiler. In this article, we will explore the concepts and algorithms involved in register allocation.

Register allocation replaces the virtual registers assigned by the compiler's code-generation module, with processor (or physical) registers.

It can be divided into three categories:

- Local register allocation: The register allocation is performed within a basic block.
- Global register allocation: That which is performed across basic blocks—i.e., over the complete function/procedure.
- Inter-procedural register allocation: That which is performed based on calling conventions, in-between functions.